

4강 파이썬 프로그래밍 3



컴퓨팅사고와 데이터 분석 기초한영신



공지사항

저희 수업을 비롯해 모든 수업의 강의영상은 해당 영상을 만들어서 업로드한 분에게 저작권이 있습니다. 강의영상을 무단으로 링크를 배포하거나 수업 외의 사용을 금합니다.

본 사이트에서 수업자료로 이용되는 저작물은 저작권법 제 25조 수업목적저작물 이용 보상금제도에 의거, 한국복제전송저작권협회와 약정을 체결하고 적법하게 이용하고 있습니다. 약정범위를 초과하는 사용은 저작권법에 저촉될 수 있으므로 수업자료의 대중 공개 공유 및 수업 목적외의 사용을 금지합니다. 해당 규정을 어기는 학생은 법적으로 처벌의 대상이 될 수 있음을 유의해 주시기 바랍니다.

본 강의교안의 저작권은 한영신입니다.

이 자료는 강의 보조자료로 제공되는 것으로 무단으로 전제하거나 배포하는 것을 금합니다 .



CONTENTS



- 01 컨테이너1
- 02 컨테이너2
- 03 파일 입출력



01 컨테이너-1



컨테이너

➤ 리스트(List)

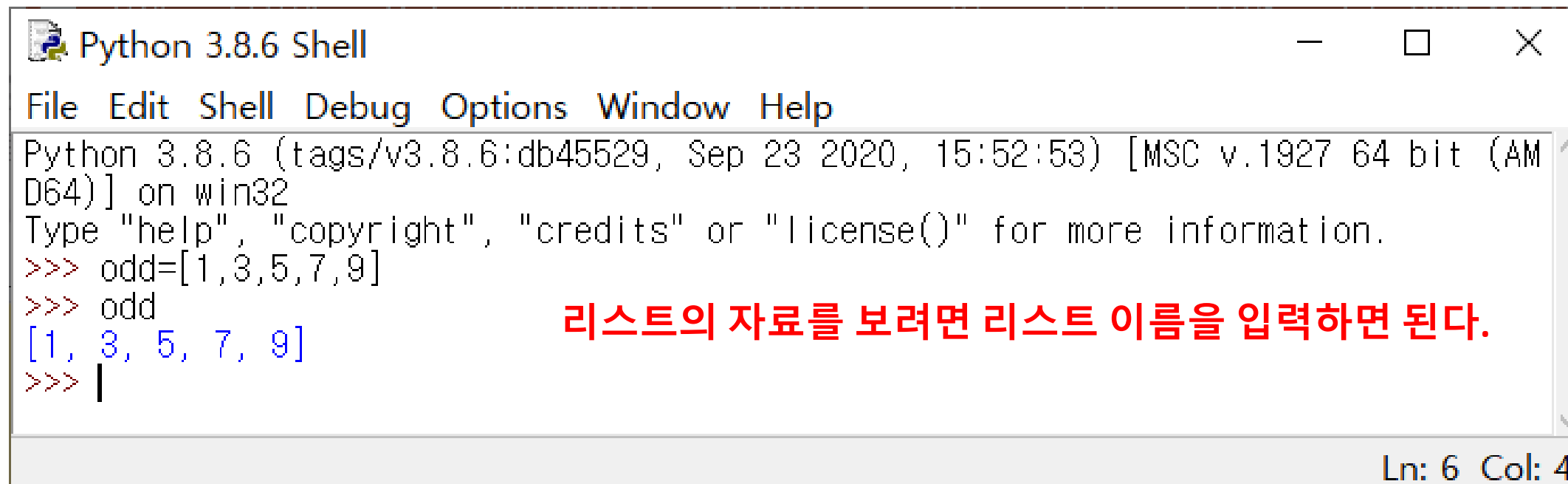
자료형의 한 종류로 숫자나 문자를 나열해서 한번에 표현

```
>>> 리스트명 = [값1, 값2, 값3, 값4,...]
```

리스트는 대괄호([])로 묶어서 표현하고 자료는 콤마(,)로 구분한다.

➤ 홀수 리스트 만들기

```
>>> odd = [1, 3, 5, 7, 9]
```



```
Python 3.8.6 Shell
File Edit Shell Debug Options Window Help
Python 3.8.6 (tags/v3.8.6:db45529, Sep 23 2020, 15:52:53) [MSC v.1927 64 bit (AMD64)] on win32
Type "help", "copyright", "credits" or "license()" for more information.
>>> odd=[1,3,5,7,9]
>>> odd
[1, 3, 5, 7, 9]
>>> |
```

리스트의 자료를 보려면 리스트 이름을 입력하면 된다.

Ln: 6 Col: 4

컨테이너 1 __ 01



컨테이너

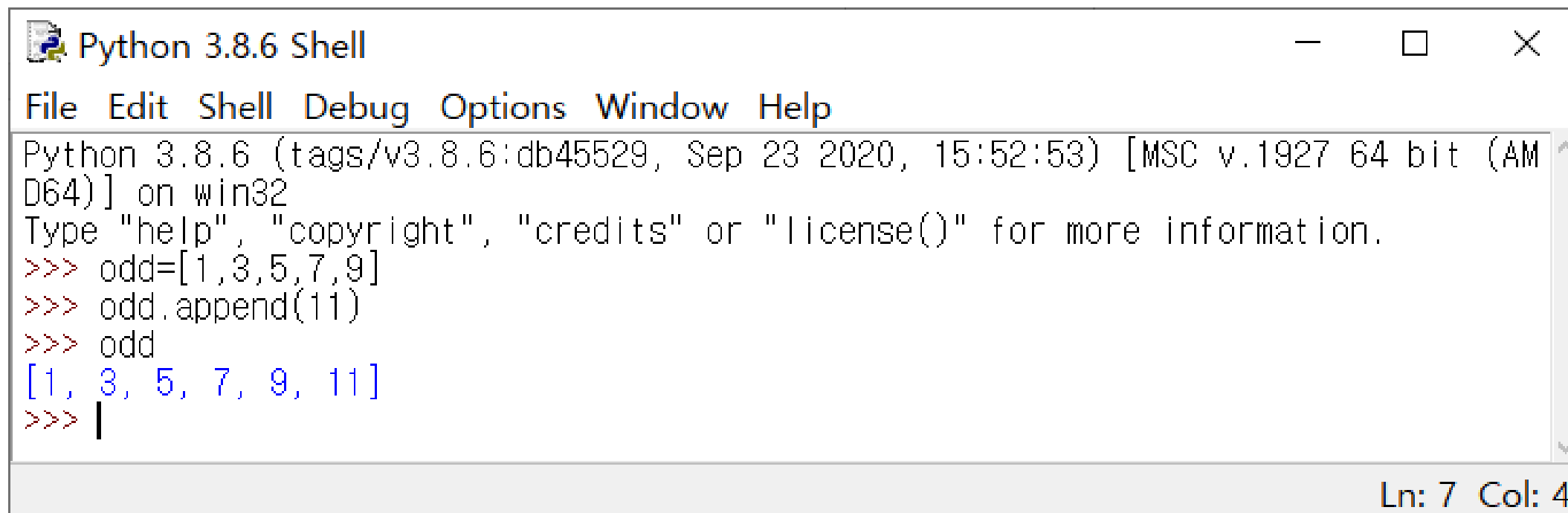
➤ 리스트(List) – append()

리스트에 자료 추가하기

```
>>> 리스트명.append(자료)
```

리스트 이름 뒤에 `.append()` 입력 후 괄호안에
추가하고 싶은 자료를 입력하면 된다.

```
>>> odd.append(11)
```



```
Python 3.8.6 Shell
File Edit Shell Debug Options Window Help
Python 3.8.6 (tags/v3.8.6:db45529, Sep 23 2020, 15:52:53) [MSC v.1927 64 bit (AMD64)] on win32
Type "help", "copyright", "credits" or "license()" for more information.
>>> odd=[1,3,5,7,9]
>>> odd.append(11)
>>> odd
[1, 3, 5, 7, 9, 11]
>>> |
```

Ln: 7 Col: 4



컨테이너

➤ 리스트(List) – insert()

특정 위치에 자료 추가하기

```
>>> 리스트명.insert(인덱스, 자료)
```

리스트 이름 뒤에 `.insert()` 입력 후 괄호안에
추가하고 싶은 자료와 인덱스를 입력하면 된다.

컨테이너 1 __ 01



추가설명

➤ 인덱스(index)

문자열과 리스트의 자료 순서를 의미한다.

```
>>> animal = ["곰", "사자", "호랑이"]
```

|
인덱스 0

|
인덱스 1

|
인덱스 2

인덱스는 0부터 시작한다.

```
Python 3.8.6 Shell
File Edit Shell Debug Options Window Help
Python 3.8.6 (tags/v3.8.6:db45529, Sep 23 2020, 15:52:53) [MSC v.1927 64 bit (AMD64)] on win32
Type "help", "copyright", "credits" or "license()" for more information.
>>> name="인하대학교"
>>> animal=["곰", "사자", "호랑이"]
>>> name[2]      '인하대학교'에서 3번째 글자
'대'
>>> animal[0]   'animal' 리스트에서 첫번째 자료
'곰'
>>> animal[2]   'animal' 리스트에서 세번째 자료
'호랑이'
>>> |
```

Ln: 11 Col: 4

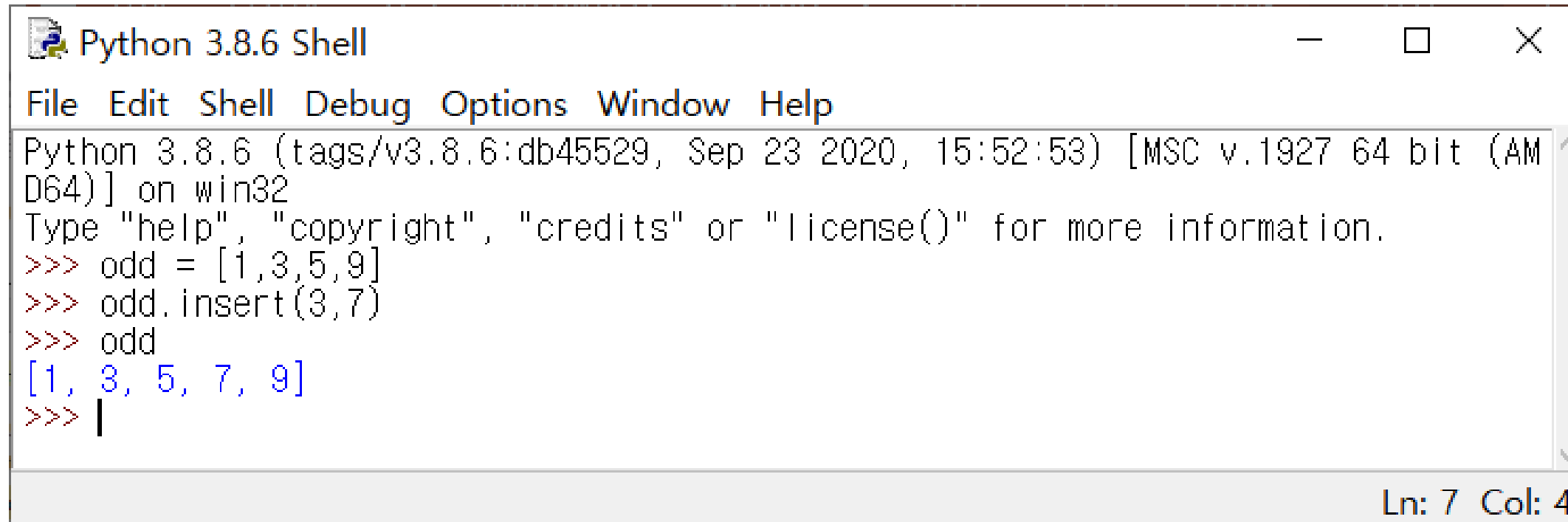


컨테이너

➤ 리스트(List) - insert

특정 위치에 자료 추가하기

>>> odd.insert(3,7) # 인덱스3에 7을 자료로 추가한다.

A screenshot of a Python 3.8.6 Shell window. The window title is "Python 3.8.6 Shell". The menu bar includes "File", "Edit", "Shell", "Debug", "Options", "Window", and "Help". The text area shows the following interaction:

```
Python 3.8.6 (tags/v3.8.6:db45529, Sep 23 2020, 15:52:53) [MSC v.1927 64 bit (AMD64)] on win32
Type "help", "copyright", "credits" or "license()" for more information.
>>> odd = [1,3,5,9]
>>> odd.insert(3,7)
>>> odd
[1, 3, 5, 7, 9]
>>> |
```

The status bar at the bottom right indicates "Ln: 7 Col: 4".

컨테이너 1 __ 01



컨테이너

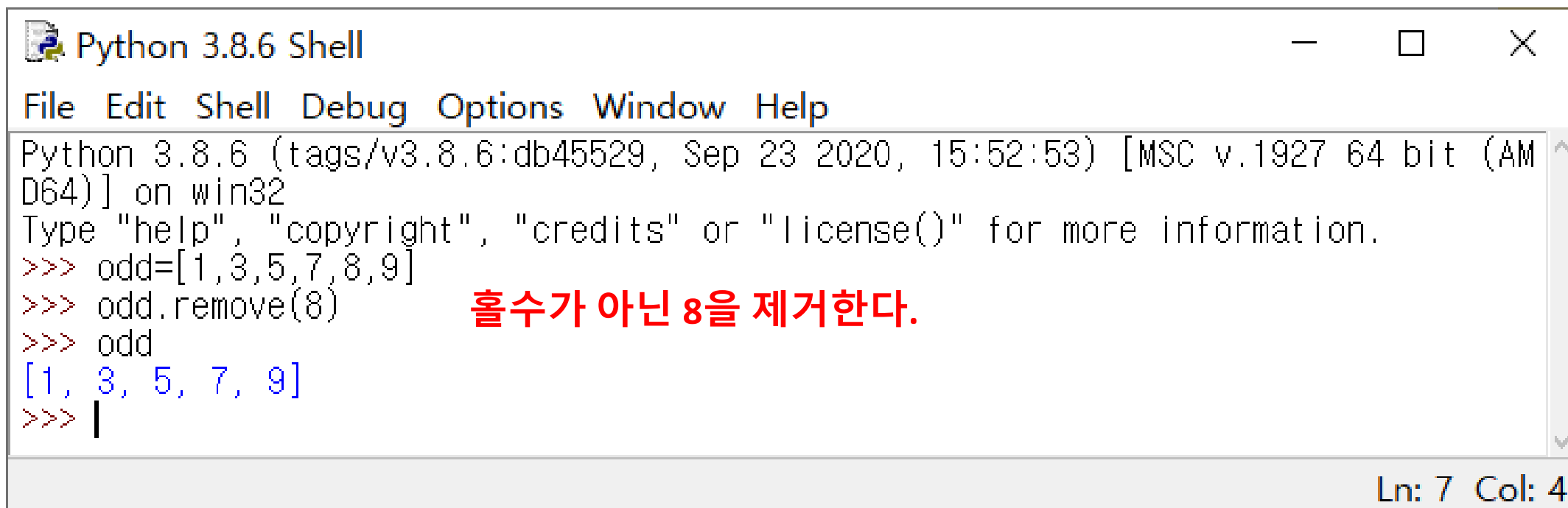
➤ 리스트(List) – remove

리스트의 자료 제거하기(1)

```
>>> 리스트명.remove(자료)
```

리스트 이름 뒤에 `.remove()` 입력 후 괄호안에
제거하고 싶은 자료를 입력하면 된다.

```
>>> odd.remove(8)
```



```
Python 3.8.6 Shell
File Edit Shell Debug Options Window Help
Python 3.8.6 (tags/v3.8.6:db45529, Sep 23 2020, 15:52:53) [MSC v.1927 64 bit (AMD64)] on win32
Type "help", "copyright", "credits" or "license()" for more information.
>>> odd=[1,3,5,7,8,9]
>>> odd.remove(8)
>>> odd
[1, 3, 5, 7, 9]
>>> |
```

홀수가 아닌 8을 제거한다.

Ln: 7 Col: 4

컨테이너 1 __ 01



컨테이너

➤ 리스트(List) – pop

리스트의 자료 제거하기(2)

```
>>> 리스트명.pop(자료)
```

리스트 이름 뒤에 **.pop()** 입력 후 괄호안에
제거하고 싶은 자료의 **인덱스**를 입력하면 된다.

pop과 remove의 차이점은

remove()는 제거하고 싶은 자료를 **직접 입력**하고

pop()는 제거하고 싶은 자료의 **인덱스**를 입력한다.

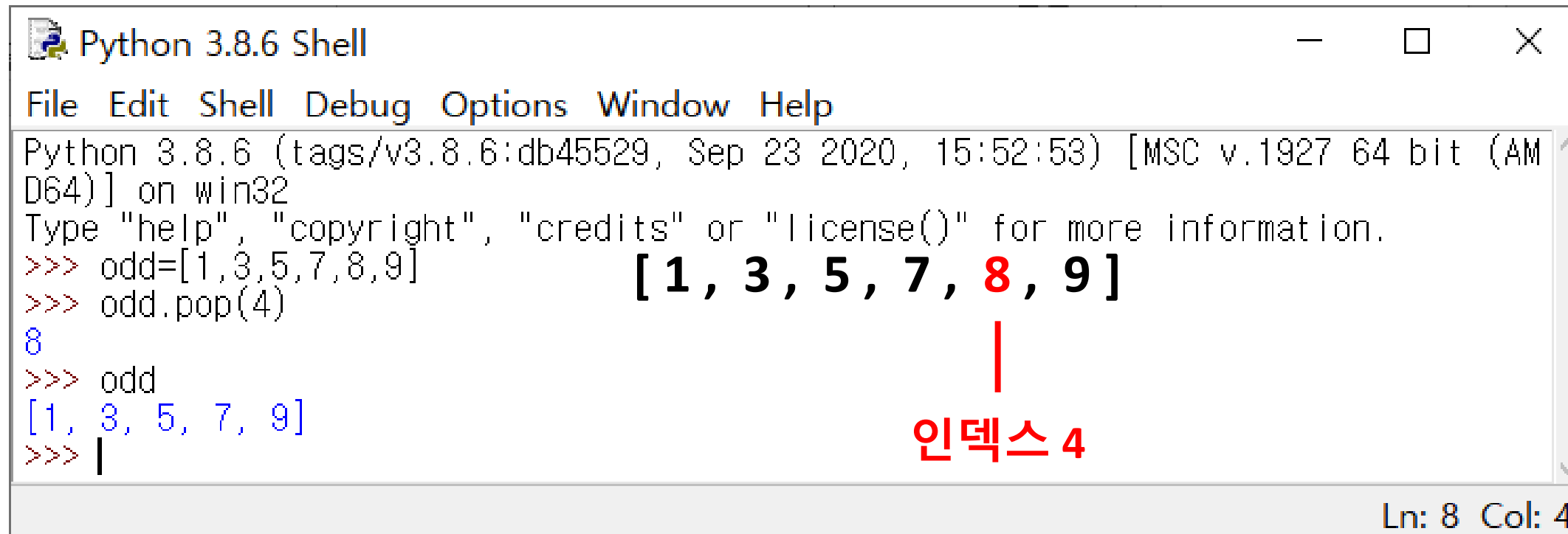


컨테이너

➤ 리스트(List) – pop

리스트의 자료 제거하기(2)

```
>>> odd.pop(4)
```



```
Python 3.8.6 Shell
File Edit Shell Debug Options Window Help
Python 3.8.6 (tags/v3.8.6:db45529, Sep 23 2020, 15:52:53) [MSC v.1927 64 bit (AMD64)] on win32
Type "help", "copyright", "credits" or "license()" for more information.
>>> odd=[1,3,5,7,8,9]
>>> odd.pop(4)
8
>>> odd
[1, 3, 5, 7, 9]
>>> |
```

The screenshot shows a Python 3.8.6 Shell window. The user has created a list `odd` with the values `[1, 3, 5, 7, 8, 9]`. The element `8` at index 4 is highlighted in red in the original image. The user then executes `odd.pop(4)`, which removes the element at index 4 and returns its value, `8`. The resulting list is `[1, 3, 5, 7, 9]`. A red vertical line and the text "인덱스 4" (Index 4) are drawn on the image to indicate the index of the removed element. The status bar at the bottom right shows "Ln: 8 Col: 4".

컨테이너 1 __ 01



컨테이너

➤ 리스트(List) – 슬라이싱

리스트의 특정 범위 가져오기

```
>>> odd[2:5]
```

0	1	2	3	4	5	6	7									
[	1	,	3	,	5	,	7	,	9	,	11	,	13	,	15	]
				[2:5]												

```
Python 3.8.6 Shell
File Edit Shell Debug Options Window Help
Python 3.8.6 (tags/v3.8.6:db45529, Sep 23 2020, 15:52:53) [MSC v.1927 64 bit (AMD64)] on win32
Type "help", "copyright", "credits" or "license()" for more information.
>>> odd=[1,3,5,7,9,11,13,15]
>>> odd[2:5]
[5, 7, 9]
>>> |
```

Ln: 6 Col: 4

컨테이너 1 __ 01



컨테이너

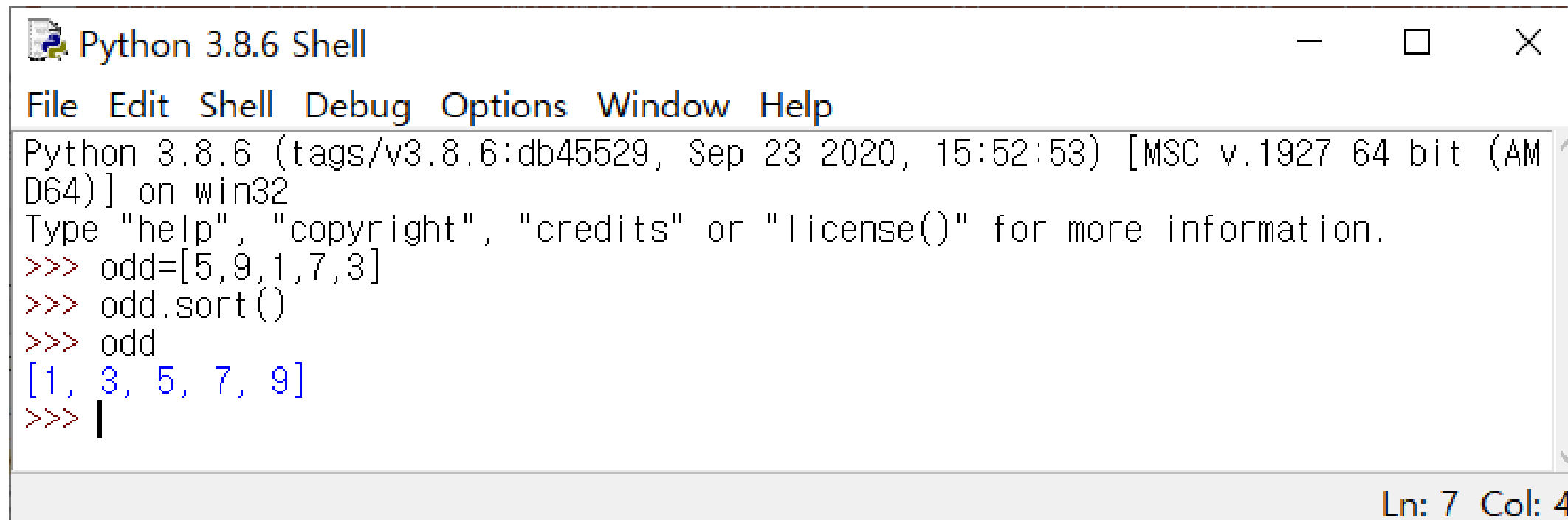
➤ 리스트(List) – sort

리스트 자료 정렬하기

```
>>> 리스트명.sort()
```

리스트 이름 뒤에 **.sort()** 입력

```
>>> odd.sort()
```



```
Python 3.8.6 Shell
File Edit Shell Debug Options Window Help
Python 3.8.6 (tags/v3.8.6:db45529, Sep 23 2020, 15:52:53) [MSC v.1927 64 bit (AMD64)] on win32
Type "help", "copyright", "credits" or "license()" for more information.
>>> odd=[5,9,1,7,3]
>>> odd.sort()
>>> odd
[1, 3, 5, 7, 9]
>>> |
```

Ln: 7 Col: 4

컨테이너 1 __ 01



컨테이너

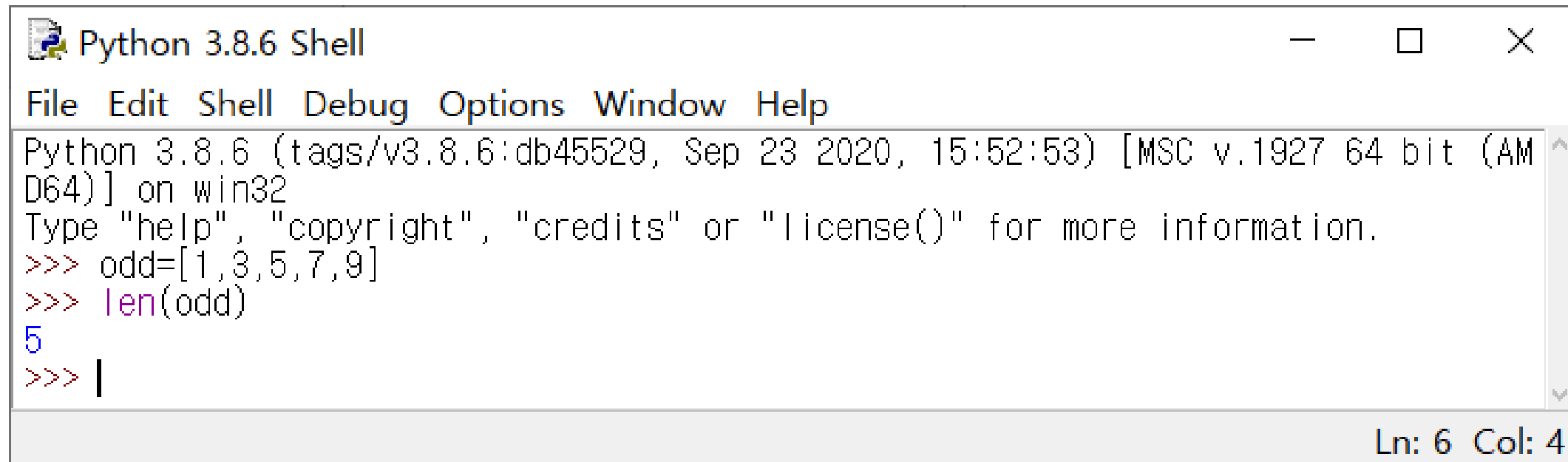
➤ 리스트(List) – len

리스트의 길이 확인하기

```
>>> len(리스트명)
```

len() 안에 리스트 이름 입력

```
>>> len(odd)
```



```
Python 3.8.6 Shell
File Edit Shell Debug Options Window Help
Python 3.8.6 (tags/v3.8.6:db45529, Sep 23 2020, 15:52:53) [MSC v.1927 64 bit (AMD64)] on win32
Type "help", "copyright", "credits" or "license()" for more information.
>>> odd=[1,3,5,7,9]
>>> len(odd)
5
>>> |
```

Ln: 6 Col: 4

컨테이너 1 __ 01



컨테이너

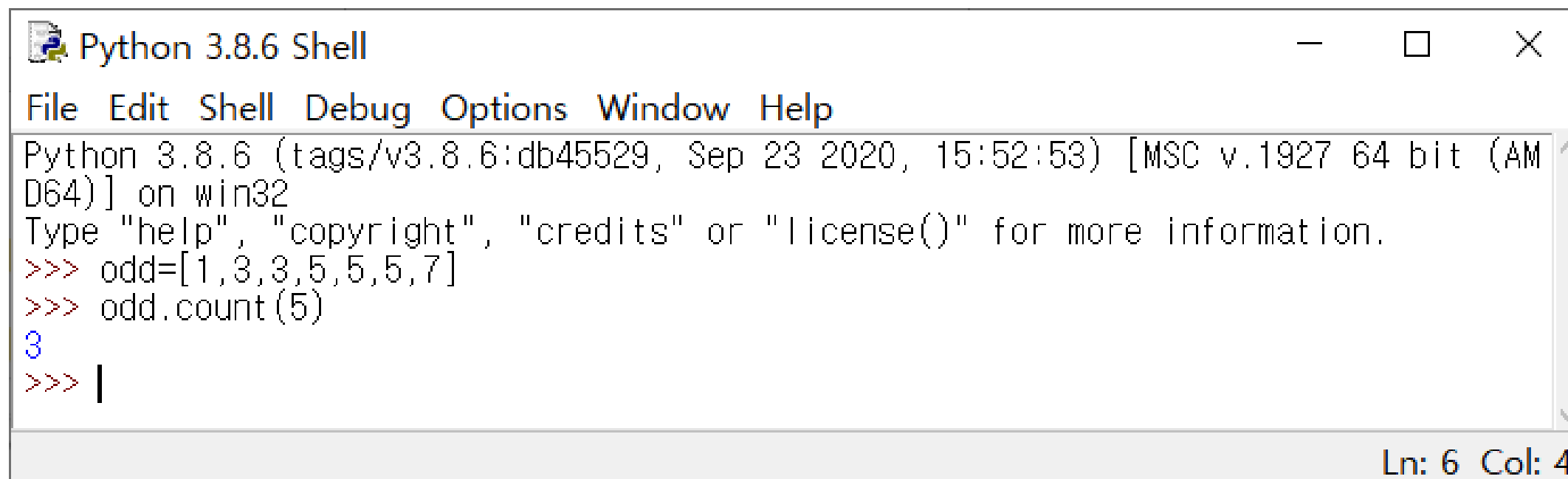
➤ 리스트(List) – count

리스트에서 찾을 값의 개수 세기

```
>>> 리스트명.count(찾을 자료)
```

리스트 이름 뒤에 `.count()` 입력 후 괄호안에
개수를 알고 싶은 값을 입력한다.

```
>>> odd.count(5)
```



```
Python 3.8.6 Shell
File Edit Shell Debug Options Window Help
Python 3.8.6 (tags/v3.8.6:db45529, Sep 23 2020, 15:52:53) [MSC v.1927 64 bit (AMD64)] on win32
Type "help", "copyright", "credits" or "license()" for more information.
>>> odd=[1,3,3,5,5,5,7]
>>> odd.count(5)
3
>>> |
```

Ln: 6 Col: 4

컨테이너 1 __ 01



컨테이너

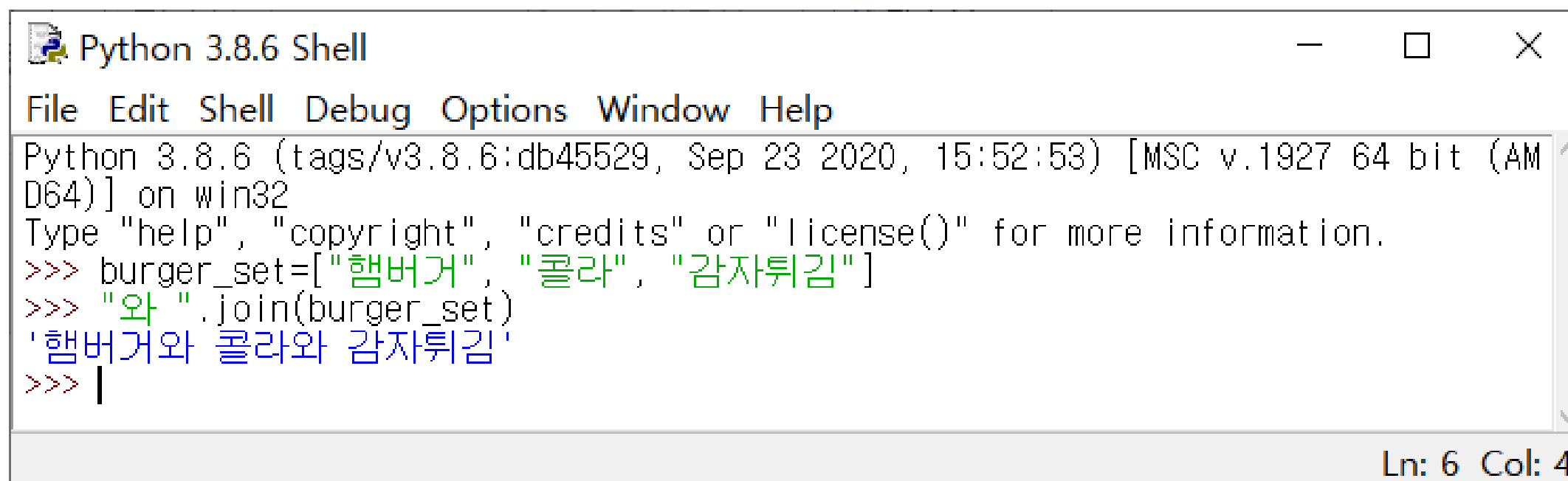
➤ 리스트(List) – join

리스트를 문자열로 바꾸기

```
>>> “단어”.join(리스트 명)
```

사이에 넣을 단어 뒤에 `.join()` 입력하고 괄호안에
문자열로 바꿀 리스트 이름을 입력하면 된다.

```
>>> “와 ”.join(burger_set)
```



```
Python 3.8.6 Shell
File Edit Shell Debug Options Window Help
Python 3.8.6 (tags/v3.8.6:db45529, Sep 23 2020, 15:52:53) [MSC v.1927 64 bit (AMD64)] on win32
Type "help", "copyright", "credits" or "license()" for more information.
>>> burger_set=["햄버거", "콜라", "감자튀김"]
>>> "와 ".join(burger_set)
'햄버거와 콜라와 감자튀김'
>>> |
```

Ln: 6 Col: 4

컨테이너 1 __ 01



실습1

컨테이너 1 __ 01

리스트를 활용해 음식 3개를 주문하기(리스트 이름: order)
3개를 입력 받으면 '짜장면, 짬뽕, 탕수육 주문되었습니다.' 같이 출력하기

```
Python 3.8.6 Shell
File Edit Shell Debug Options Window Help
Python 3.8.6 (tags/v3.8.6:db45529, Sep 23 2020, 15:52:53) [MSC v.1927 64 bit (AMD64)] on win32
Type "help", "copyright", "credits" or "license()" for more information.
>>>
===== RESTART: D:\python.py =====
주문하고 싶은 음식을 입력하세요짜장면
주문내역: ['짜장면']
주문하고 싶은 음식을 입력하세요짬뽕
주문내역: ['짜장면', '짬뽕']
주문하고 싶은 음식을 입력하세요탕수육
주문내역: ['짜장면', '짬뽕', '탕수육']
짜장면, 짬뽕, 탕수육 주문 되었습니다.
>>> |
```

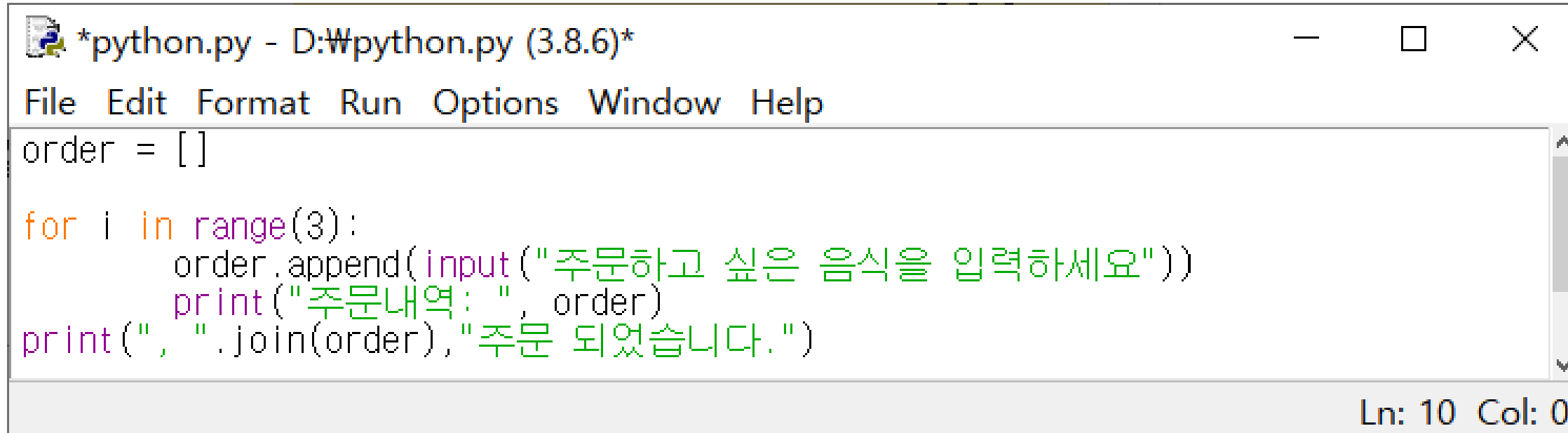
Ln: 12 Col: 4



실습1

컨테이너 1 __ 01

리스트를 활용해 음식 3개를 주문하기(리스트 이름: order)
3개를 입력 받으면 '짜장면, 짬뽕, 탕수육 주문되었습니다.' 같이 출력하기



```
*python.py - D:\Wpython.py (3.8.6)*
File Edit Format Run Options Window Help
order = []

for i in range(3):
    order.append(input("주문하고 싶은 음식을 입력하세요"))
    print("주문내역: ", order)
print(", ".join(order), "주문 되었습니다.")

Ln: 10 Col: 0
```

02 컨테이너-2



컨테이너

➤ 튜플(Tuple)

튜플은 한번 저장된 값을 수정할 수 없는 자료형이다.

```
>>> 튜플명 = (값1, 값2, 값3, 값4, ...)
```

➤ 튜플(Tuple)의 특징

튜플은 리스트와 다르게 괄호(())로 표현한다.

괄호를 사용하지 않아도 된다(tuple = 1, 2, 3)

저장된 데이터를 변경할 수 없다는 점만 제외하면 리스트와 동일하다.

컨테이너 2 __ 02



컨테이너

➤ 딕셔너리(Dictionary)

순서가 없는 컬렉션 자료형으로 각각의 요소는 key:value 형태로 저장된다.

```
>>> 딕셔너리명 =  
      {key1:value1, key2:value2, key3:value3, ...}
```

➤ 딕셔너리(Dictionary)의 특징

- 딕셔너리는 리스트나 튜플처럼 index에 의해 해당 요소를 찾지 않고 key를 통해 value를 얻는다.
- {}안에 키:값 형식의 항목을 콤마(,)로 구분해 하나 이상 저장할 수 있는 컬렉션 자료형이다.
- 딕셔너리는 키를 먼저 지정하고 :(콜론)을 붙여서 값을 표현한다. 키와 값은 1:1 대응관계이다.

컨테이너 2 __ 02



컨테이너

➤ 딕셔너리(Dictionary)

```
>>> menu =  
{'김밥':2000, '라면':3000, '돈까스':5000}
```

키(key)

값(value)

김밥



2000

라면



3000

돈까스



5000

컨테이너 2 __ 02

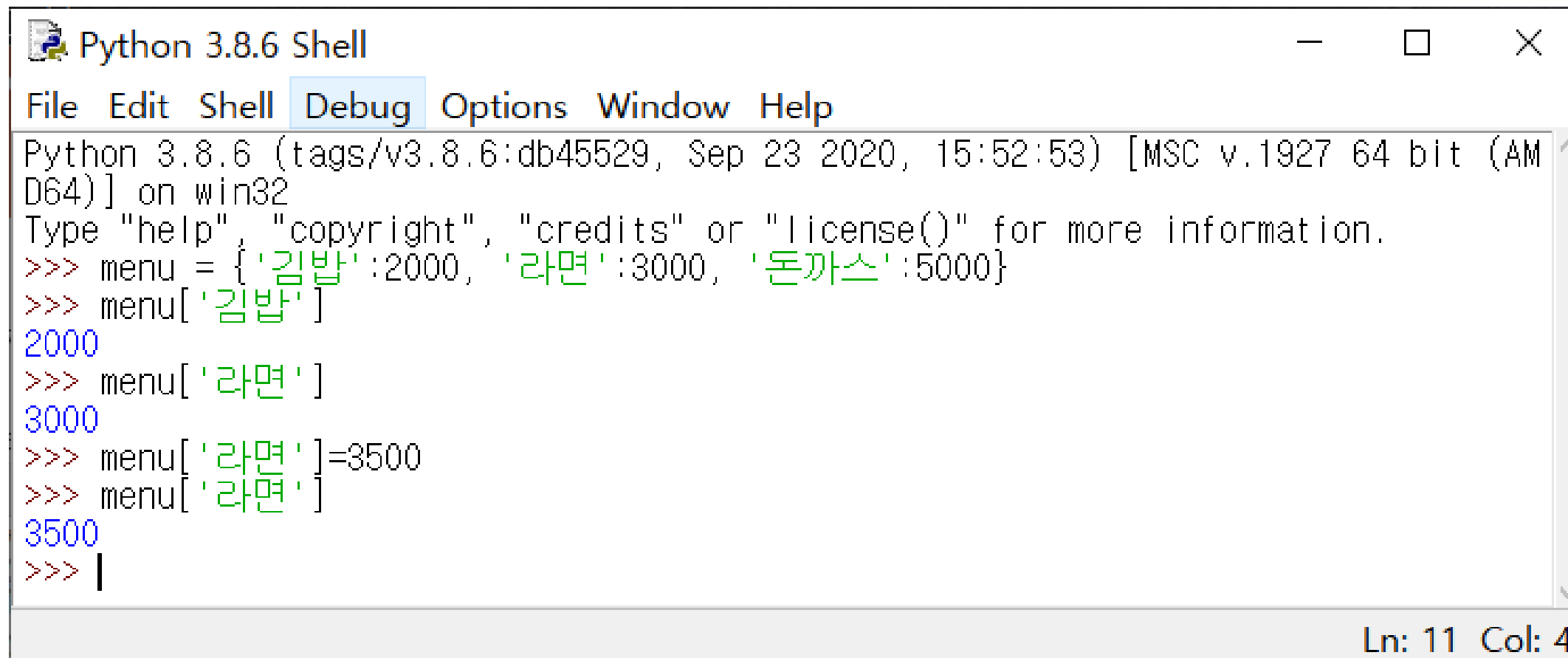


컨테이너

➤ 딕셔너리(Dictionary)

딕셔너리 요소에 접근하기

>>> 딕셔너리명[key]



```
Python 3.8.6 Shell
File Edit Shell Debug Options Window Help
Python 3.8.6 (tags/v3.8.6:db45529, Sep 23 2020, 15:52:53) [MSC v.1927 64 bit (AMD64)] on win32
Type "help", "copyright", "credits" or "license()" for more information.
>>> menu = {'김밥':2000, '라면':3000, '돈까스':5000}
>>> menu['김밥']
2000
>>> menu['라면']
3000
>>> menu['라면']=3500
>>> menu['라면']
3500
>>> |
```

Ln: 11 Col: 4

리스트와 다르게 인덱스 대신 **key**를 입력한다.

컨테이너 2 __ 02



컨테이너

➤ 딕셔너리(Dictionary)

모든 key, value, item 반환하기 – keys, values, items

```
>>> 딕셔너리명.keys()      # 키들만 모아서 반환
>>> 딕셔너리명.values()    # 값들만 모아서 반환
>>> 딕셔너리명.items()     # key와 value의 쌍을 튜플로 모아서 반환
```

```
Python 3.8.6 Shell
File Edit Shell Debug Options Window Help
Python 3.8.6 (tags/v3.8.6:db45529, Sep 23 2020, 15:52:53) [MSC v.1927 64 bit (AMD64)] on win32
Type "help", "copyright", "credits" or "license()" for more information.
>>> menu = {'김밥':2000, '라면':3000, '돈까스':5000}
>>> menu.keys()
dict_keys(['김밥', '라면', '돈까스'])
>>> menu.values()
dict_values([2000, 3000, 5000])
>>> menu.items()
dict_items([('김밥', 2000), ('라면', 3000), ('돈까스', 5000)])
>>> |
```

Ln: 10 Col: 4

컨테이너 2 __ 02



컨테이너

➤ 딕셔너리(Dictionary)

딕셔너리 요소 삭제하기 – del, pop

```
>>> del(딕셔너리명[key])  
>>> 딕셔너리명.pop(key)
```

```
Python 3.8.6 Shell  
File Edit Shell Debug Options Window Help  
Python 3.8.6 (tags/v3.8.6:db45529, Sep 23 2020, 15:52:53) [MSC v.1927 64 bit (AMD64)] on win32  
Type "help", "copyright", "credits" or "license()" for more information.  
>>> menu = {'김밥':2000, '라면':3000, '돈까스':5000}  
>>> del(menu['김밥'])  
>>> menu  
{'라면': 3000, '돈까스': 5000}  
>>> menu.pop('돈까스')  
5000  
>>> menu  
{'라면': 3000}  
>>> |
```

Ln: 11 Col: 4

컨테이너 2 __ 02



컨테이너

➤ 세트(Set)

중복을 허용하지 않는 컬렉션 자료형이다.

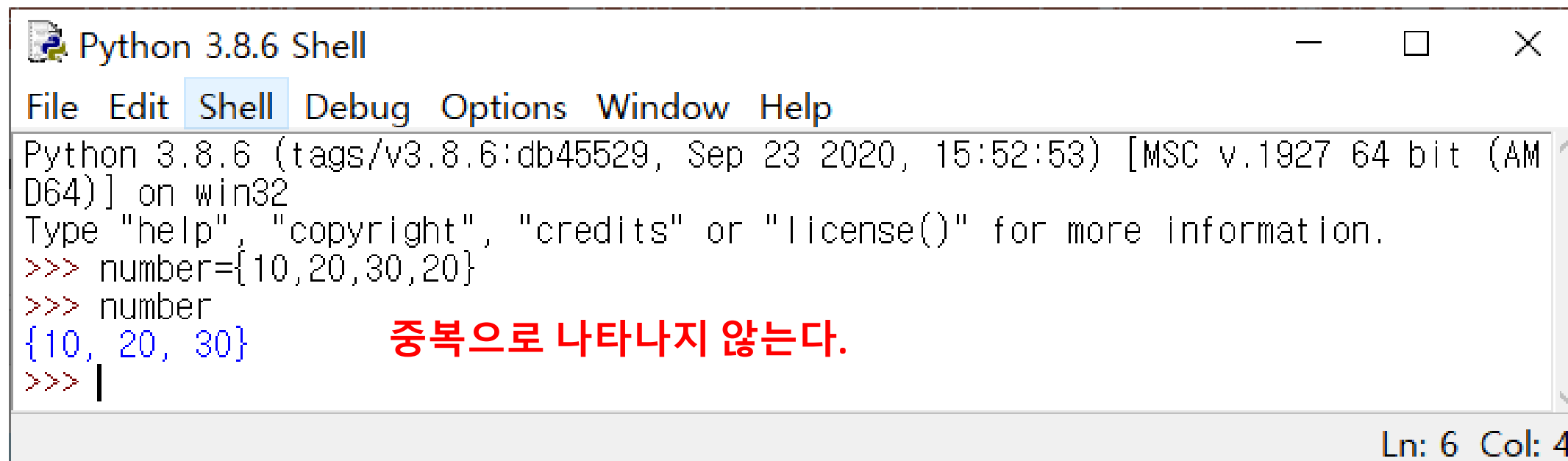
>>> 세트명 = {값1, 값2, 값3, 값4,...}

➤ 세트(Set)의 특징

수학의 집합과 같다.

중복되지 않은 요소들로 구성된다.

세트 자료형은 순서가 없다.(인덱싱 불가능)



```
Python 3.8.6 Shell
File Edit Shell Debug Options Window Help
Python 3.8.6 (tags/v3.8.6:db45529, Sep 23 2020, 15:52:53) [MSC v.1927 64 bit (AMD64)] on win32
Type "help", "copyright", "credits" or "license()" for more information.
>>> number={10,20,30,20}
>>> number
{10, 20, 30}
>>> |
```

중복으로 나타나지 않는다.

Ln: 6 Col: 4

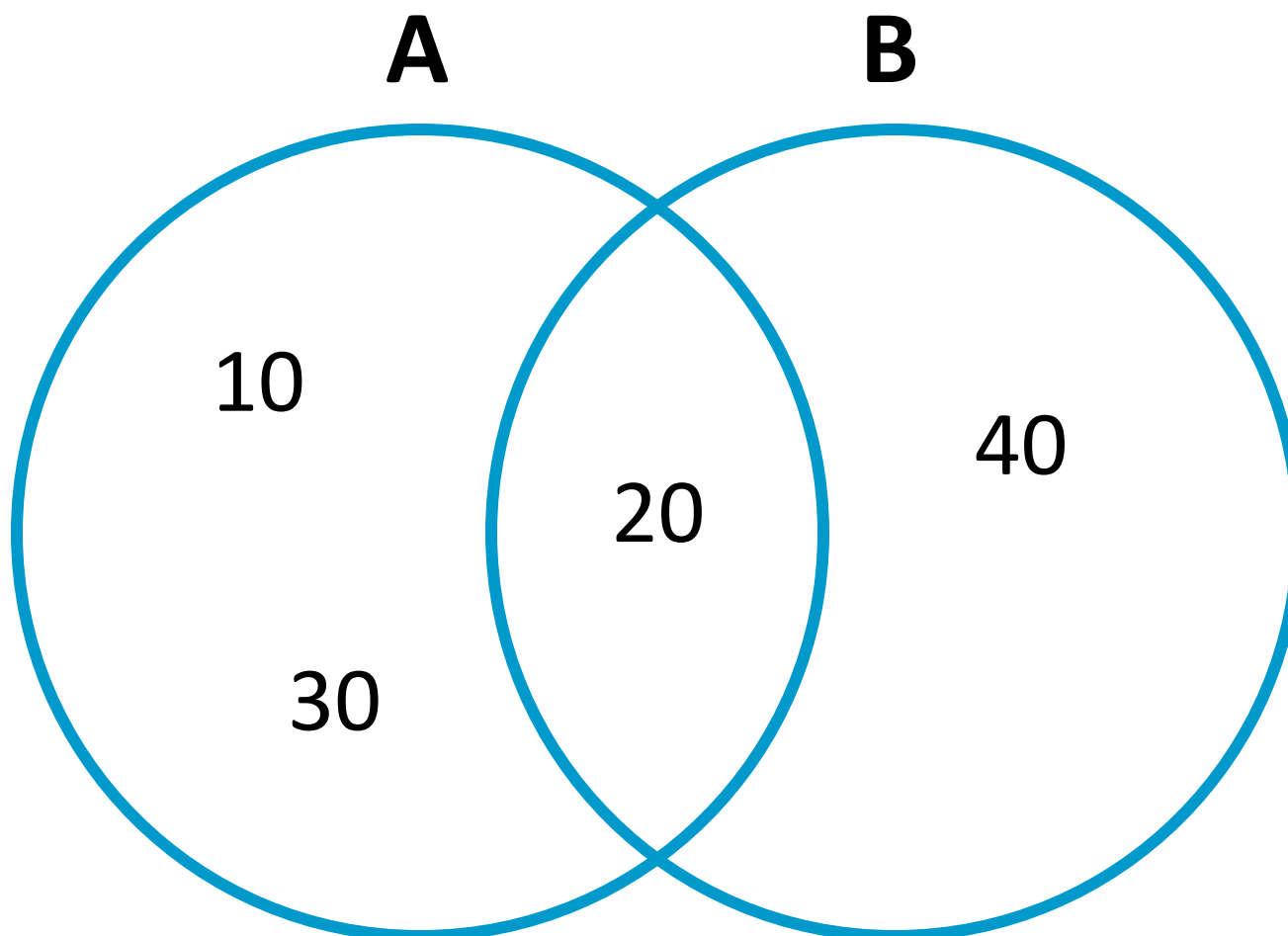
컨테이너 2 __ 02



컨테이너

▶ 세트(Set)

```
*Python 3.8.6 Shell*
File Edit Shell Debug Options Window Help
Python 3.8.6 (tags/v3.8.6:db45529, Sep 23 2020, 15:52:53) [MSC v.1927 64 bit (AMD64)] on win32
Type "help", "copyright", "credits" or "license()" for more information.
>>> A={10,20,30}
>>> B={20,40}
```



컨테이너 2 __ 02



컨테이너

▶ 세트(Set)

교집합, 합집합, 차집합 – intersection, union, difference

```
Python 3.8.6 Shell
File Edit Shell Debug Options Window Help
>>> A={10,20,30}
>>> B={20,40}
>>> A & B                                #교집합
{20}
>>> A.intersection(B)                   #교집합
{20}
>>> A | B                                #합집합
{20, 40, 10, 30}
>>> A.union(B)                           #합집합
{20, 40, 10, 30}
>>> A - B                                #차집합
{10, 30}
>>> A.difference(B)                     #차집합
{10, 30}
>>> |
Ln: 17 Col: 4
```

컨테이너 2 __ 02



컨테이너

➤ 세트(Set)

값 추가와 삭제 – add, update, remove

>>> 세트명.add(자료)	# 값 1개추가
>>> 세트명.update([자료1,자료2])	# 값 여러 개 추가
>>> 세트명.remove(자료)	# 특정 값 삭제

```
Python 3.8.6 Shell
File Edit Shell Debug Options Window Help
Python 3.8.6 (tags/v3.8.6:db45529, Sep 23 2020, 15:52:53) [MSC v.1927 64 bit (AMD64)] on win32
Type "help", "copyright", "credits" or "license()" for more information.
>>> A={10,20,30}
>>> A.add(40)
>>> A
{40, 10, 20, 30}
>>> A.update([50,60])
>>> A
{40, 10, 50, 20, 60, 30}
>>> A.remove(40)
>>> A
{10, 50, 20, 60, 30}
>>> |
```

Ln: 13 Col: 4

컨테이너 2 __ 02



03

파일 입출력



파일 입출력

▶ 파일의 개념

- 실행중인 프로그램이 종료되면 처리한 결과 값은 메모리에서 사라지기 때문에 모든 데이터는 더 이상 사용할 수가 없다.
- 따라서 프로그램을 실행하는 도중에 데이터를 영구적으로 저장하고 싶다면 보조기억장치(하드디스크)에 파일 형태로 저장해야 한다.

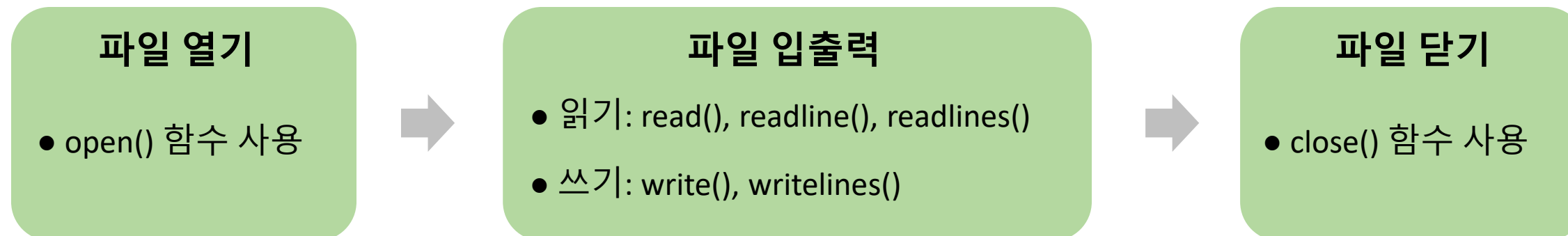
파일 입출력 __ 03



파일 입출력

▶ 파일의 개념

- 파일에 저장된 데이터를 읽고 처리하는 방법은 제일 먼저 사용하려는 파일을 `open()` 함수를 통해 열어야 한다. 파일이 열리면 파일에 있는 데이터를 읽거나 쓸 수 있다. 그리고 파일과 관련된 작업이 모두 종료되면 파일을 `close()` 함수를 통해 닫아주는 것이 좋다.



파일 입출력 __ 03



파일 입출력

▶ 파일열기 - open

현재 디렉토리(파이썬 파일이 저장된 위치)에 있는 파일을 열고 파일을 생성하기 위해 파이썬 내장 함수 open()를 사용

```
>>> 파일객체 = open(파일명, 파일열기모드)
```

open() 함수는 다음과 같이 “파일 이름”과 “파일 열기 모드”를 입력값으로 받고 결과값으로 파일 객체를 반환한다.

```
>>> 변수명 = open('파일명', 'r')      # 읽기모드
>>> 변수명 = open('파일명', 'w')      # 쓰기모드
>>> 변수명 = open('파일명', 'a')      # 추가모드
```

파일 입출력 __ 03



파일 입출력

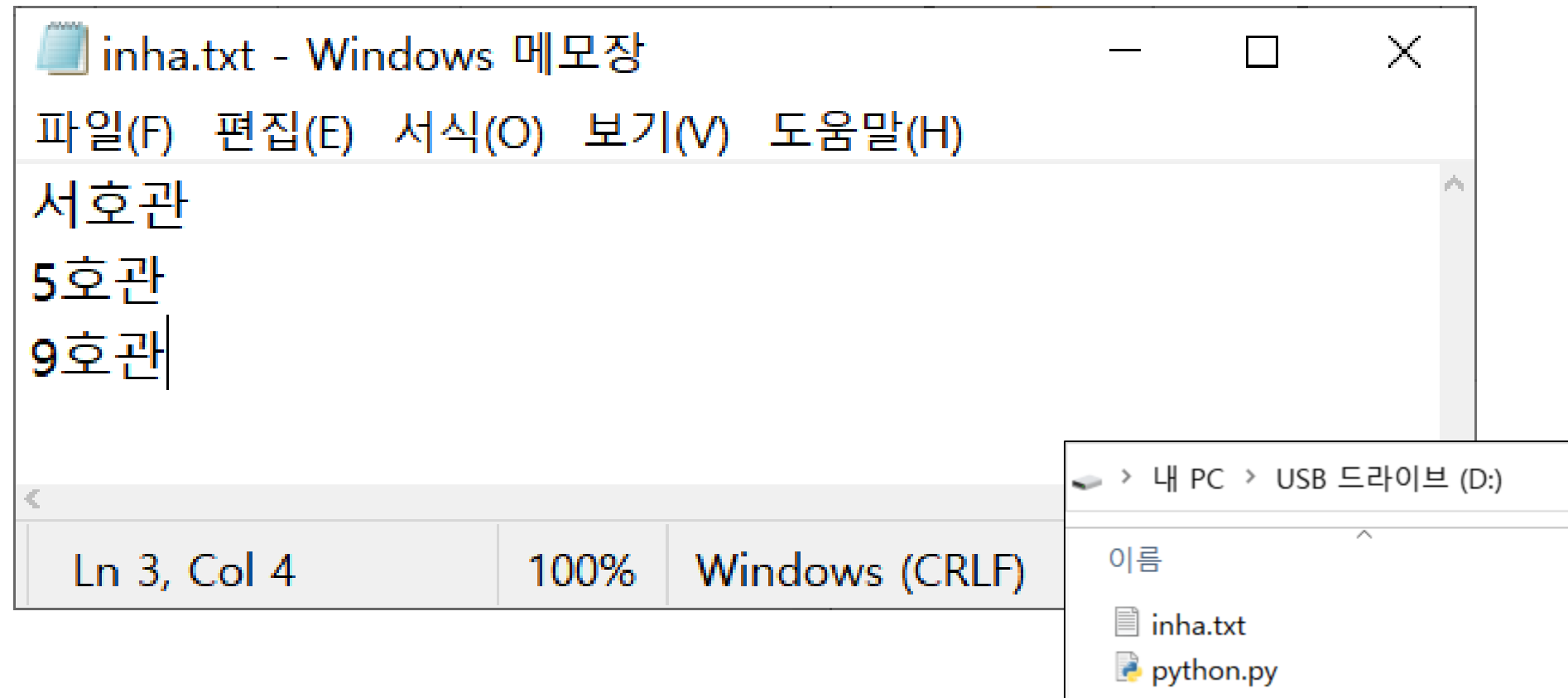
➤ 텍스트 파일 읽기 - read

read() 함수는 파일의 전체 내용을 문자열로 반환한다.

```
>>> 변수 = 파일객체.read()
```

데이터를 읽기를 실행하기 위한 텍스트 파일을 생성해야 한다.

현재 디렉토리(같은 파일 경로)에 메모장을 이용해 'inha.txt' 생성하자.



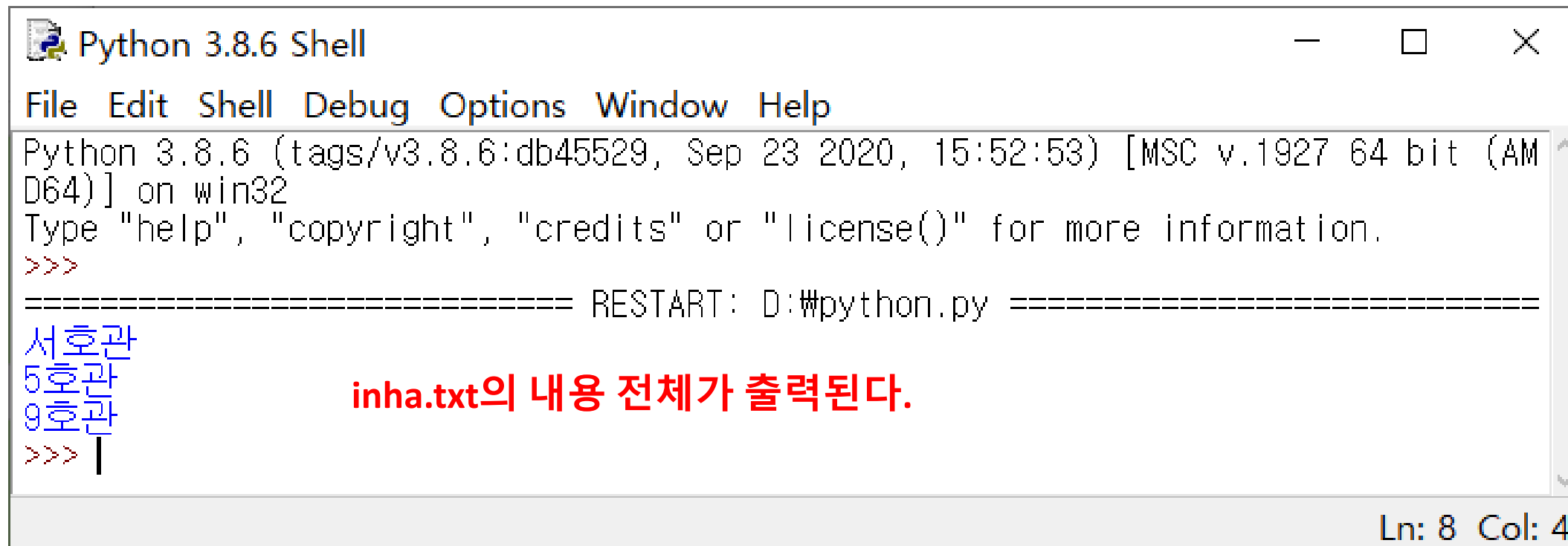
파일 입출력 __ 03



파일 입출력

▶ 텍스트 파일 읽기 - read

```
text = open('inha.txt', 'r')      # 파일열기
data = text.read()                # 텍스트 파일 읽기
print(data)                       #파일닫기
text.close()
```



```
Python 3.8.6 Shell
File Edit Shell Debug Options Window Help
Python 3.8.6 (tags/v3.8.6:db45529, Sep 23 2020, 15:52:53) [MSC v.1927 64 bit (AMD64)] on win32
Type "help", "copyright", "credits" or "license()" for more information.
>>>
===== RESTART: D:\#python.py =====
서호관
5호관
9호관
>>> |
```

Ln: 8 Col: 4

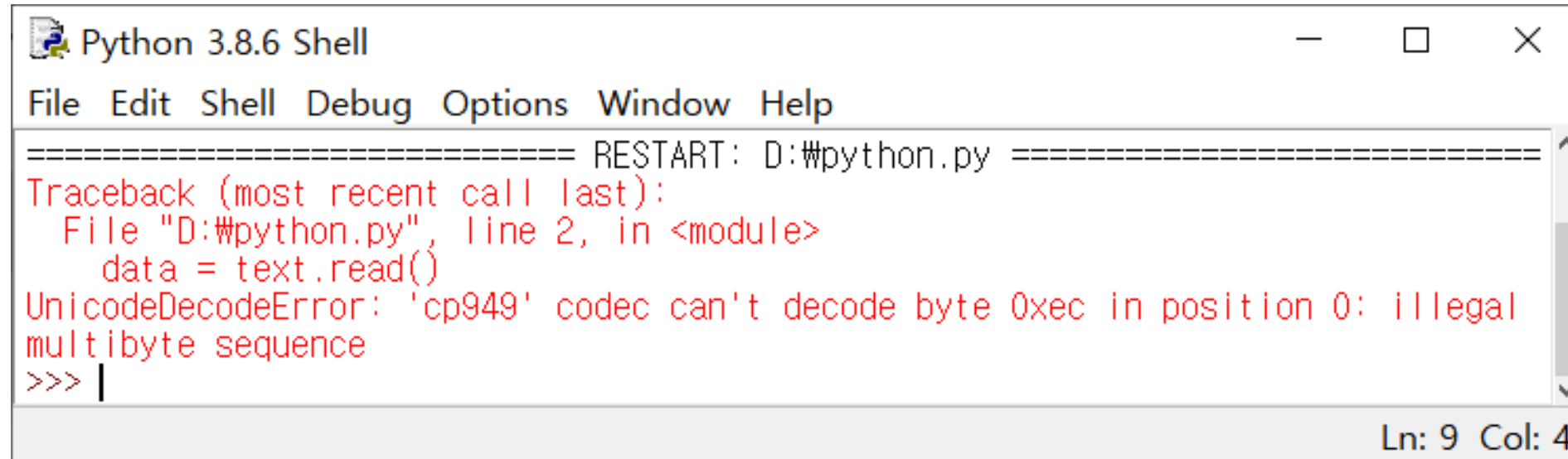
inha.txt의 내용 전체가 출력된다.

파일 입출력 __ 03



파일 입출력

➤ 오류



```
Python 3.8.6 Shell
File Edit Shell Debug Options Window Help
===== RESTART: D:\#python.py =====
Traceback (most recent call last):
  File "D:\#python.py", line 2, in <module>
    data = text.read()
UnicodeDecodeError: 'cp949' codec can't decode byte 0xec in position 0: illegal
multibyte sequence
>>> |
Ln: 9 Col: 4
```

위와 같은 **오류**가 뜨면

```
text = open('inha.txt', 'r')
```

```
→ text = open('inha.txt', 'r', encoding='UTF8')
```

로 수정해 보세요.

파일 입출력 __ 03



파일 입출력

➤ 텍스트 파일 읽기 – readline

readline() 함수는 파일의 내용을 한 줄씩 읽어서 문자열로 반환한다

```
>>> 변수 = 파일객체.readline()
```

```
text = open('inha.txt', 'r')           # 파일열기
data = text.readline()                 # 텍스트 파일 읽기
print(data)
text.close()                           #파일닫기
```

```
===== RESTART: D:\python.py =====
```

서호관

파일 입출력 __ 03



파일 입출력

➤ 텍스트 파일 읽기 – readlines

readline() 함수는 한 줄 씩 읽지만, readlines() 함수는 파일의 모든 줄을 읽어서 리스트로 반환한다.

```
>>> 변수 = 파일객체.readlines()
```

```
text = open('inha.txt', 'r')          # 파일열기
data = text.readlines()               # 텍스트 파일 읽기
Print(data)
Text.close()                          #파일닫기
```

```
===== RESTART: D:\python.py =====
['서호관', '5호관', '9호관']
```

파일 입출력 __ 03



파일 입출력

➤ 텍스트 파일 쓰기 -write

프로그램에서 처리한 결과 값을 파일에 저장하기 위해서는 open() 함수로 파일을 쓰기 모드('w')로 생성한 후 다양한 함수(write(), writelines())를 이용한다

>>> 파일객체.write(내용)

```
text=open('inha.txt', 'w')           # 쓰기 모드로 파일 열기
while True:
    building = input('building:')      # building 변수 입력
    if not building:break              # 입력되지 않으면 반복 종료
    text.write(building)               # 파일에 변수 쓰기
text.close()
```

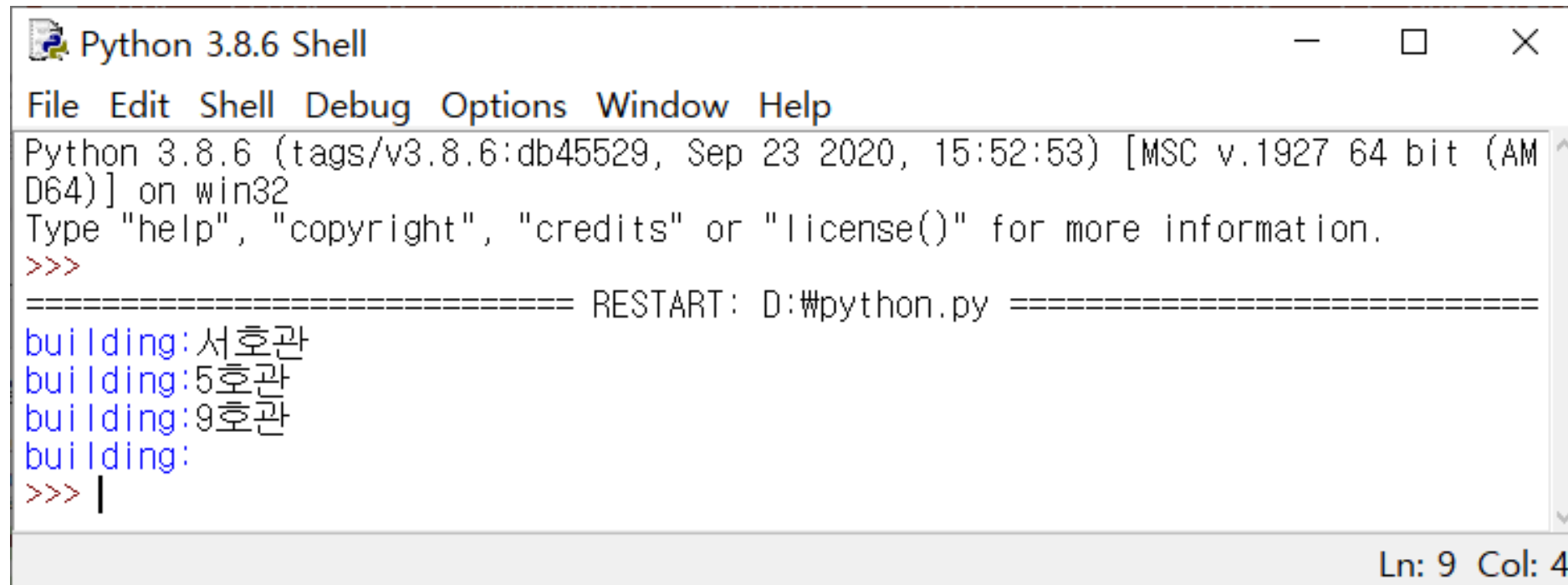
파일 입출력 __ 03



파일 입출력

파일 입출력 __ 03

▶ 텍스트 파일 쓰기 -write

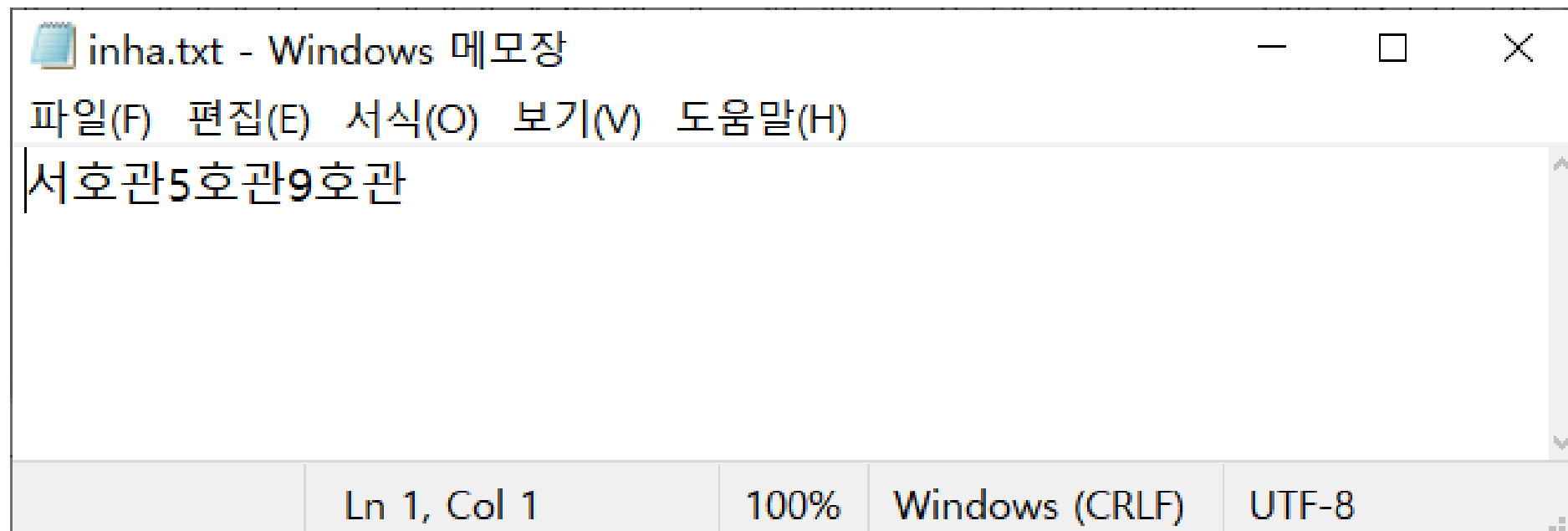


```
Python 3.8.6 Shell
File Edit Shell Debug Options Window Help
Python 3.8.6 (tags/v3.8.6:db45529, Sep 23 2020, 15:52:53) [MSC v.1927 64 bit (AMD64)] on win32
Type "help", "copyright", "credits" or "license()" for more information.
>>>
===== RESTART: D:\python.py =====
building:서호관
building:5호관
building:9호관
building:
>>> |
```

Ln: 9 Col: 4



inha.txt 파일이 존재하지 않았어도 생성된다.



```
inha.txt - Windows 메모장
파일(F) 편집(E) 서식(O) 보기(V) 도움말(H)
서호관5호관9호관
```

Ln 1, Col 1 100% Windows (CRLF) UTF-8

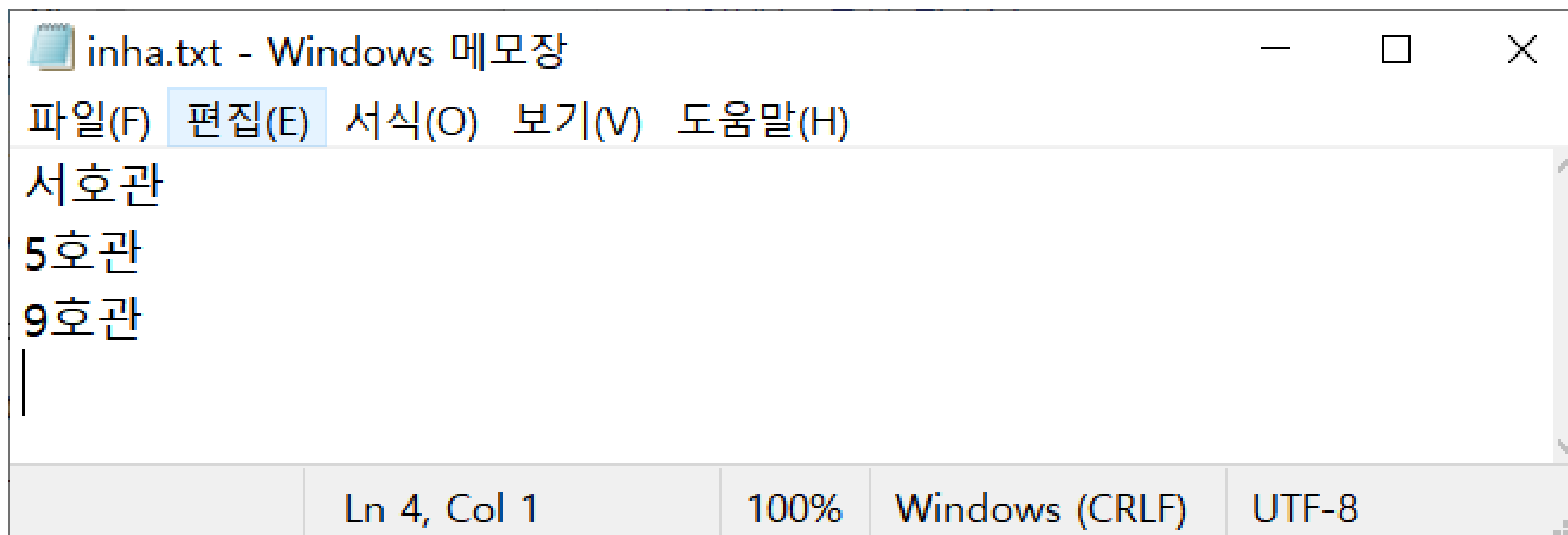


파일 입출력

➤ 텍스트 파일 쓰기 -write

개행을 하고싶다면 개행 문자('\n')를 포함하면 된다.

```
text=open('inha.txt', 'w')
while True:
    building = input('building:')
    if not building:break
    text.write(building + '\n')          # 개행문자 추가( '\ ' = 'W' )
Text.close()
```



파일 입출력 __ 03



파일 입출력

➤ 텍스트 파일 쓰기 – 추가모드(a)

파일 쓰기 모드('w')로 파일을 열 때는 기존 파일의 내용이 모두 사라진다.
기존 파일 내용을 유지하며 새로운 값을 추가해 하는 경우 추가모드('a')를 사용한다.

```
text=open('inha.txt', 'a')           # 추가 모드로 파일 열기
while True:
    building = input('building:')
    if not building:break
    text.write(building + '\n')
Text.close()
```

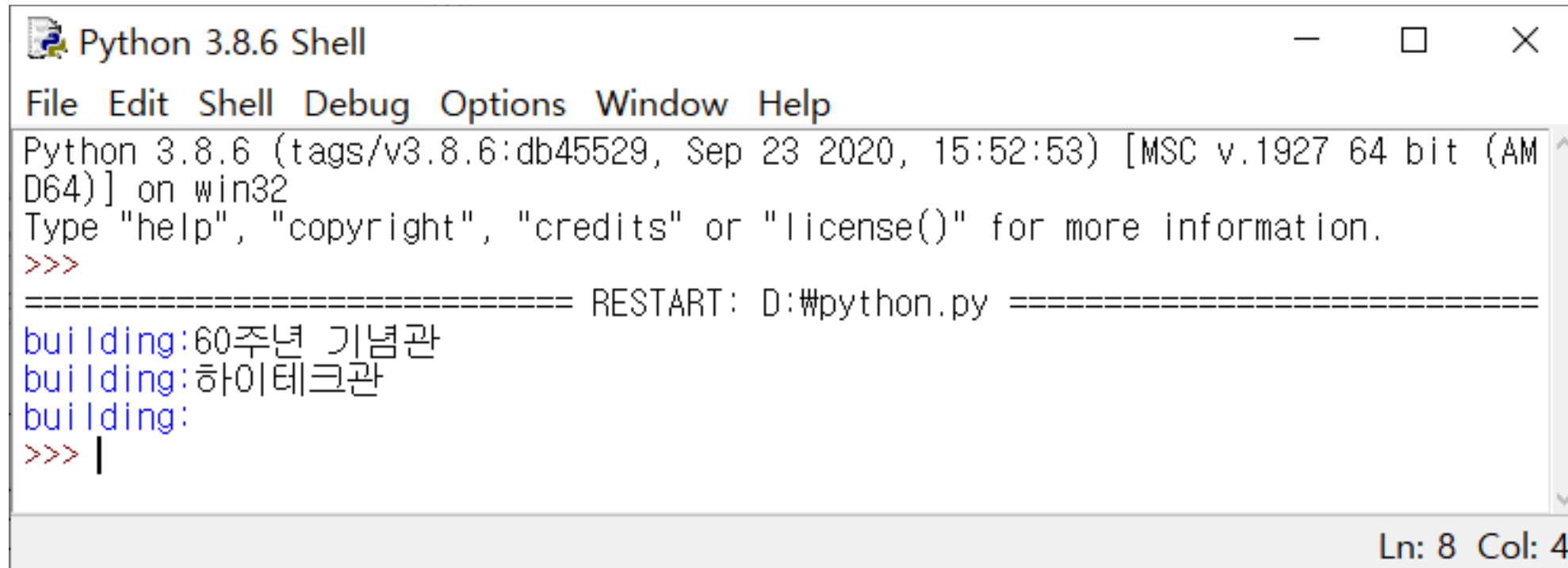
파일 입출력 __ 03



파일 입출력

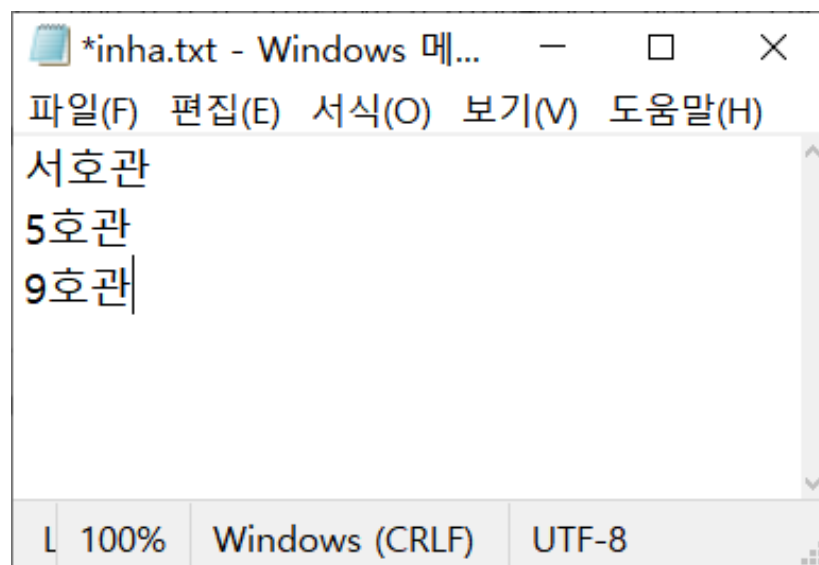
파일 입출력 __ 03

▶ 텍스트 파일 쓰기 - 추가모드(a)



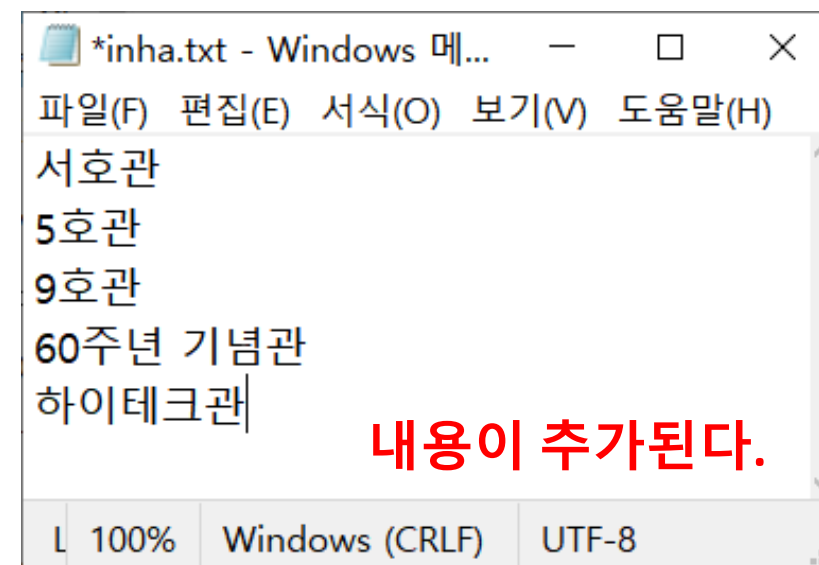
```
Python 3.8.6 Shell
File Edit Shell Debug Options Window Help
Python 3.8.6 (tags/v3.8.6:db45529, Sep 23 2020, 15:52:53) [MSC v.1927 64 bit (AMD64)] on win32
Type "help", "copyright", "credits" or "license()" for more information.
>>>
===== RESTART: D:\#python.py =====
building:60주년 기념관
building:하이테크관
building:
>>> |
```

Ln: 8 Col: 4



```
*inha.txt - Windows 메...
파일(F) 편집(E) 서식(O) 보기(V) 도움말(H)
서호관
5호관
9호관
```

L 100% Windows (CRLF) UTF-8



```
*inha.txt - Windows 메...
파일(F) 편집(E) 서식(O) 보기(V) 도움말(H)
서호관
5호관
9호관
60주년 기념관
하이테크관
```

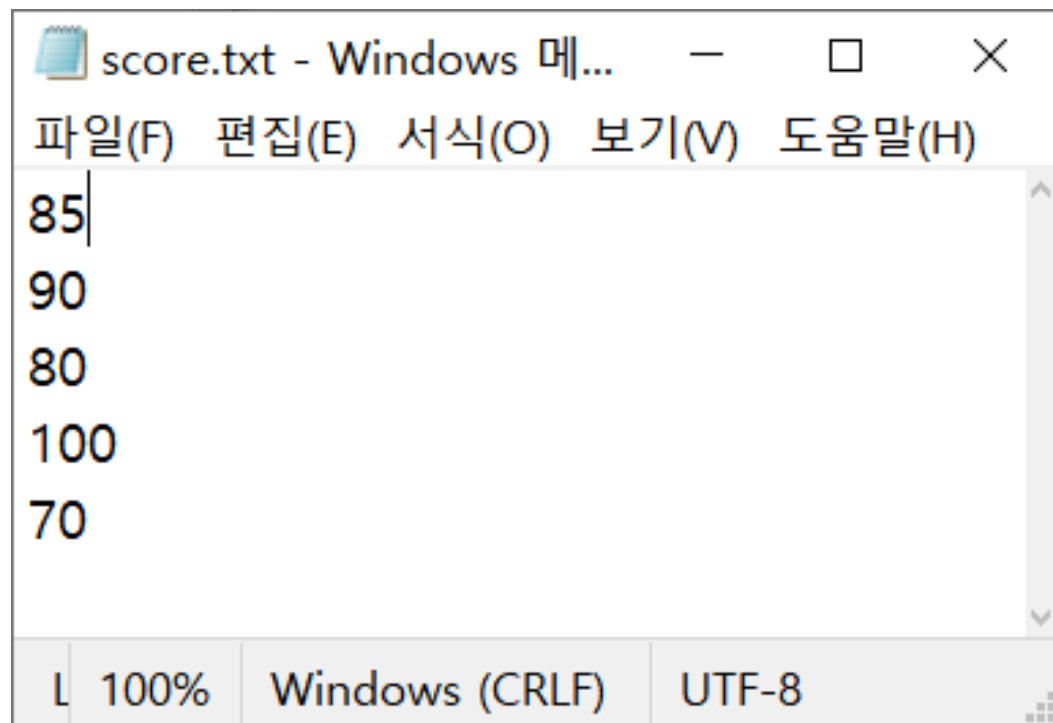
L 100% Windows (CRLF) UTF-8

내용이 추가된다.

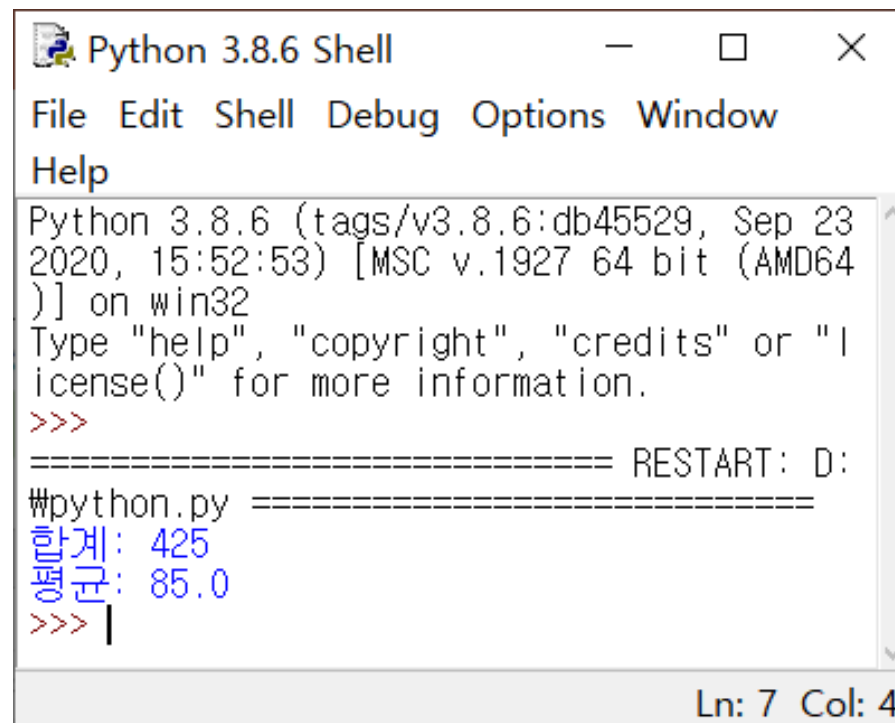


실습 예제

- # 점수가 쓰여진 텍스트 파일을 불러와 평균 구하기
- # 메모장 파일 이름(score.txt)
- # 파일객체(math), 합(sum), 평균(avg)



```
score.txt - Windows 메모장
파일(F) 편집(E) 서식(O) 보기(V) 도움말(H)
85
90
80
100
70
L 100% Windows (CRLF) UTF-8
```



```
Python 3.8.6 Shell
File Edit Shell Debug Options Window Help
Python 3.8.6 (tags/v3.8.6:db45529, Sep 23 2020, 15:52:53) [MSC v.1927 64 bit (AMD64)] on win32
Type "help", "copyright", "credits" or "license()" for more information.
>>>
===== RESTART: D: =====
#python.py
합계: 425
평균: 85.0
>>>
Ln: 7 Col: 4
```

파일 입출력 __ 03



실습 예제

➤ 분석하기

```
math=open('score.txt')  
lines=math.readlines()
```

```
sum = 0
```

```
# 파일을 읽기 모드로 불러온다  
# 모든 줄을 리스트로 반환
```

```
# sum 초기값 0
```

실습 예제

➤ 분석하기

```
math=open('score.txt')
lines=math.readlines()

sum = 0

for line in lines:
    line = line.rstrip()
    score=int(line)
    sum += score
```

line을 lines(5)만큼 반복
개행 문자 제거
score에 데이터 저장
sum = sum + score

실습 예제

➤ 분석하기

```
math=open('score.txt')
lines=math.readlines()

sum = 0

for line in lines:
    line = line.rstrip()
    score=int(line)
    sum += score

avg=sum/len(lines)                # len(lines) = 5

print('합계:', sum)                # 합계 출력
print('평균:', avg)                # 평균 출력

math.close()                      # 파일닫기
```


Thank you

컴퓨팅 사고와 데이터분석 기초
한영신

